شرح سطر بسطر ٦ — شاشة 01: الترحيب

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

الإطار في (844×390) `مرحبًا 01` :Figma. تتكوّن من قسمين: «هيرو» داكن في الأعلى بزوايا سفلية مدوّرة، وقسم محتوى فيه ثلاث نقاط قيمة ونقاط صفحات وزر ورابط.

متى تظهر؟ فقط عندما لا توجد ملفات على الجهاز. مسارها `/welcome`. إن وُجد ملف واحد بلا رمز سري يدخل الطالب الرئيسية مباشرة دون أن يراها.

الاتجاه RTL: أول عنصر في `Row` على اليمين، لذلك المربع الملوّن يسبق النص في `_ValuePoint`، والأيقونة تسبق الكلمة في الزر.

lib/features/onboarding/welcome_screen.dart — شاشة الترحيب

lib/features/onboarding/welcome_screen.dart

الأسطر 1–4

```
1   import 'package:flutter/material.dart';
2   import 'package:flutter/services.dart';
3   import 'package:flutter_svg/flutter_svg.dart';
4   import 'package:go_router/go_router.dart';
```

ماذا يفعل؟ استيراد Material وخدمات النظام وSVG وgo_router.

لماذا بهذه الطريقة؟ `services.dart` فيها `SystemUiOverlayStyle` لتلوين أيقونات شريط الحالة.

الأسطر 6–12

```
6   import '../../app/routes.dart';
7   import '../../core/constants/app_assets.dart';
8   import '../../core/strings/ar.dart';
9   import '../../core/theme/app_colors.dart';
10  import '../../core/theme/app_text_styles.dart';
11  import '../../core/widgets/icon_tile.dart';
12  import '../../core/widgets/primary_button.dart';
```

ماذا يفعل؟ استيراد المسارات والأصول والنصوص والألوان والأنماط ومكوّنات الأيقونة والزر.

لماذا بهذه الطريقة؟ لا نستورد الشاشة التالية؛ ننتقل بالاسم فقط عبر `AppRoutes`.

الأسطر 14–16

```
14  /// الشاشة 01 — مرحبًا
15  class WelcomeScreen extends StatelessWidget {
16    const WelcomeScreen({super.key});
```

ماذا يفعل؟ تعليق وصنف `WelcomeScreen` بلا حالة.

لماذا بهذه الطريقة؟ الشاشة ثابتة لا تحتفظ بأي بيانات.

الأسطر 18–19

```
18    @override
19    Widget build(BuildContext context) {
```

ماذا يفعل؟ دالة `build`.

لماذا بهذه الطريقة؟ تعيد شجرة الواجهة.

الأسطر 20–21

```
20      return AnnotatedRegion<SystemUiOverlayStyle>(
21        value: SystemUiOverlayStyle.light,
```

ماذا يفعل؟ `AnnotatedRegion` يجعل أيقونات شريط الحالة فاتحة.

لماذا بهذه الطريقة؟ الهيرو داكن، فلو بقيت الساعة والبطارية داكنة لاختفت. `SystemUiOverlayStyle.light` = أيقونات بيضاء.

السطر 22

```
22        child: Scaffold(
```

ماذا يفعل؟ `Scaffold` هيكل الصفحة.

لماذا بهذه الطريقة؟ يأخذ لون الخلفية `ground` من الثيم.

السطر 23

```
23          body: SingleChildScrollView(
```

ماذا يفعل؟ `SingleChildScrollView` للمحتوى.

لماذا بهذه الطريقة؟ الأجهزة الصغيرة أو الخط الكبير قد تتجاوز 844؛ التمرير يمنع خطأ «overflow».

الأسطر 24–27

```
24            child: Column(
25              crossAxisAlignment: CrossAxisAlignment.stretch,
26              children: [
27                const _Hero(),
```

ماذا يفعل؟ `Column` يمدّ أبناءه بعرض الشاشة ويضع الهيرو أولًا.

لماذا بهذه الطريقة؟ `stretch` يعطي الهيرو العرض الكامل.

الأسطر 28–32

```
28                Padding(
29                  padding: const EdgeInsets.symmetric(
30                    horizontal: 24,
31                    vertical: 28,
32                  ),
```

ماذا يفعل؟ حشوة للمحتوى: 24 أفقيًا و28 عموديًا.

لماذا بهذه الطريقة؟ قيم Figma.

الأسطر 33–36

```
33                child: Column(
34                  crossAxisAlignment: CrossAxisAlignment.stretch,
35                  spacing: 18,
36                  children: [
```

ماذا يفعل؟ `Column` داخلي يمدّ العناصر مع مسافة 18.

لماذا بهذه الطريقة؟ `spacing` تغني عن `SizedBox(height: 18)` بين كل عنصرين.

الأسطر 37–42

```
37                    const _ValuePoint(
38                      asset: AppAssets.bookTile,
39                      tileColor: AppColors.tealTint,
40                      title: ArStrings.welcomeValue1Title,
41                      subtitle: ArStrings.welcomeValue1Subtitle,
42                    ),
```

ماذا يفعل؟ نقطة القيمة الأولى: أيقونة كتاب بخلفية `tealTint` وعنوان ووصف من `ArStrings`.

لماذا بهذه الطريقة؟ ثلاث نقاط بنفس التركيب فصنعنا لها ودجتًا واحدًا `_ValuePoint`.

الأسطر 43–48

```
43                    const _ValuePoint(
44                      asset: AppAssets.cloudOff,
45                      tileColor: AppColors.blueTint,
46                      title: ArStrings.welcomeValue2Title,
47                      subtitle: ArStrings.welcomeValue2Subtitle,
48                    ),
```

ماذا يفعل؟ النقطة الثانية: السحابة المشطوبة بخلفية زرقاء (التعلّم دون إنترنت).

لماذا بهذه الطريقة؟ اللون الأزرق يميّز ميزة «دون اتصال».

الأسطر 49–54

```
49                    const _ValuePoint(
50                      asset: AppAssets.target,
51                      tileColor: AppColors.amberTint,
52                      title: ArStrings.welcomeValue3Title,
53                      subtitle: ArStrings.welcomeValue3Subtitle,
54                    ),
```

ماذا يفعل؟ النقطة الثالثة: الهدف بخلفية عنبرية (المراجعة).

لماذا بهذه الطريقة؟ العنبري يعبّر عن «انتباه/مراجعة».

السطر 55

```
55                    const _PageDots(count: 3, current: 0),
```

ماذا يفعل؟ نقاط الصفحات: 3 والحالية الأولى.

لماذا بهذه الطريقة؟ التصميم يوحي بثلاث شاشات تعريفية؛ الودجت جاهز لذلك.

الأسطر 56–60

```
                    PrimaryButton(
                      label: ArStrings.welcomeStart,
                      iconAsset: AppAssets.fwdOnboarding,
                      onPressed: () => context.push(AppRoutes.createStep1),
                    ),
```

ماذا يفعل؟ زر «ابدأ» بأيقونة سهم؛ عند الضغط ينتقل إلى `createStep1` بـ `push`.

لماذا بهذه الطريقة؟ `push` وليس `go` ليستطيع الطالب الرجوع إلى الترحيب. الأيقونة هي أول عنصر فتظهر يمين النص.

الأسطر 61–67

```
                    TextButton(
                      onPressed: () =>
                          ScaffoldMessenger.of(context).showSnackBar(
                            const SnackBar(
                              content: Text(ArStrings.restoreNeedsInternet),
                            ),
                          ),
```

ماذا يفعل؟ رابط «لديّ ملف طالب على هذا الجهاز»: يعرض `SnackBar` برسالة أن الاستعادة تحتاج إنترنتًا.

لماذا بهذه الطريقة؟ الاستعادة برمز الطالب تحتاج خادمًا غير موجود بعد؛ نقول الحقيقة بدل أن نزيّف نجاحًا.

الأسطر 68–73

```
                      style: TextButton.styleFrom(
                        foregroundColor: AppColors.teal,
                        textStyle: AppTextStyles.body15,
                      ),
                      child: const Text(ArStrings.welcomeHaveProfile),
                    ),
```

ماذا يفعل؟ نمط الرابط: لون teal ونص `body15`، والنص من `ArStrings`.

لماذا بهذه الطريقة؟ `TextButton` لأنه إجراء ثانوي لا يستحق ثِقل الزر الرئيسي. حجم اللمس الافتراضي (48) مناسب لإمكانية الوصول.

الأسطر 74–83

```
                  ],
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس والدالة والصنف.

لماذا بهذه الطريقة؟ نهاية `WelcomeScreen`.

الأسطر 85–88

```
class _Hero extends StatelessWidget {
  const _Hero();

  @override
```

ماذا يفعل؟ ودجت `_Hero` خاص بلا حالة.

لماذا بهذه الطريقة؟ الشرطة السفلية تجعله خاصًّا بالملف. قسم مستقل ومعقّد فنفصله.

الأسطر 89–95

```
  Widget build(BuildContext context) {
    return Container(
      clipBehavior: Clip.antiAlias,
      decoration: const BoxDecoration(
        color: AppColors.ink,
        borderRadius: BorderRadius.vertical(bottom: Radius.circular(36)),
      ),
```

ماذا يفعل؟ `Container` بلون `ink` وزوايا سفلية مدوّرة 36 مع `clipBehavior`.

لماذا بهذه الطريقة؟ شريط التطريز أعرض (408) من الشاشة (390) فنقصّه عند الحواف بدل أن يتجاوزها.

الأسطر 96–97

```
      child: SafeArea(
        bottom: false,
```

ماذا يفعل؟ `SafeArea` بلا حماية من الأسفل.

لماذا بهذه الطريقة؟ نحمي شريط الحالة من الأعلى ليبدأ الشريط تحته.

الأسطر 98–99

```
        child: Padding(
          padding: const EdgeInsets.only(bottom: 40),
```

ماذا يفعل؟ حشوة سفلية 40.

لماذا بهذه الطريقة؟ مسافة بين النص والحافة المدوّرة.

الأسطر 100–101

```
          child: Column(
            children: [
```

ماذا يفعل؟ `Column` أبناؤه في الوسط.

لماذا بهذه الطريقة؟ التوسّط الأفقي هو المطلوب.

الأسطر 102–107

```
102              SvgPicture.asset(
103                AppAssets.tatreezBand,
104                width: double.infinity,
105                height: 24,
106                fit: BoxFit.cover,
107              ),
```

ماذا يفعل؟ شريط التطريز: عرض كامل وارتفاع 24 و`BoxFit.cover`.

لماذا بهذه الطريقة؟ الأصل بعرض 408؛ `cover` يملأ عرض أي شاشة ويقصّ الزائد فيطابق التصميم على 390 ولا يترك فراغًا على الأعرض.

الأسطر 108–109

```
108              Padding(
109                padding: const EdgeInsetsDirectional.fromSTEB(24, 32, 24, 0),
```

ماذا يفعل؟ حشوة: 24 يمينًا ويسارًا و32 من الأعلى (بصيغة `Directional`).

لماذا بهذه الطريقة؟ من Figma.

الأسطر 110–112

```
110                child: Column(
111                  spacing: 14,
112                  children: [
```

ماذا يفعل؟ `Column` بمسافة 14 بين الأبناء.

لماذا بهذه الطريقة؟ مسافة التصميم بين الشخصية والشعار والنص.

الأسطر 113–117

```
113                    SvgPicture.asset(
114                      AppAssets.mascot150,
115                      width: 150,
116                      height: 150,
117                    ),
```

ماذا يفعل؟ الشخصية بحجم 150.

لماذا بهذه الطريقة؟ نستعمل ملف SVG الأصلي بدون تلوين لأن هذه الشاشة قبل اختيار اللون.

الأسطر 118–121

```
118                    Row(
119                      mainAxisSize: MainAxisSize.min,
120                      spacing: 10,
121                      children: [
```

ماذا يفعل؟ `Row` بحجم أدنى ومسافة 10 للشعار والاسم.

لماذا بهذه الطريقة؟ يتوسّط الصف داخل العمود.

الأسطر 122–126

```
                    SvgPicture.asset(
                      AppAssets.logoOnDark,
                      width: 40,
                      height: 40,
                    ),
```

ماذا يفعل؟ شعار «رِزمة» على الداكن بحجم 40.

لماذا بهذه الطريقة؟ هو أول عنصر فيظهر يمين الكلمة كما في Figma.

الأسطر 127–130

```
                    Text(
                      ArStrings.appName,
                      style: AppTextStyles.wordmark40,
                    ),
```

ماذا يفعل؟ كلمة «رِزمة» من `ArStrings` بنمط `wordmark40`.

لماذا بهذه الطريقة؟ التشكيل موجود في النص كما في التصميم.

الأسطر 131–132

```
                  ],
                ),
```

ماذا يفعل؟ إغلاق الـ Row.

لماذا بهذه الطريقة؟ نهاية الصف.

الأسطر 133–137

```
                Text(
                  ArStrings.welcomeTagline,
                  textAlign: TextAlign.center,
                  style: AppTextStyles.tagline18,
                ),
```

ماذا يفعل؟ الجملة التعريفية بنمط `tagline18` ومحاذاة للمنتصف.

لماذا بهذه الطريقة؟ تلتفّ على سطرين كما في التصميم (ارتفاع السطر 1.6).

الأسطر 138–147

```
                ],
              ),
            ),
          ],
        ),
      ),
    ),
  );
 }
}
```

ماذا يفعل؟ إغلاق الأقواس والدالة والصنف.

لماذا بهذه الطريقة؟ نهاية `_Hero`.

الأسطر 149–155

```
class _ValuePoint extends StatelessWidget {
 const _ValuePoint({
  required this.asset,
  required this.tileColor,
  required this.title,
  required this.subtitle,
 });
```

ماذا يفعل؟ ودجت `_ValuePoint`: الأيقونة ولون الخلفية والعنوان والوصف.

لماذا بهذه الطريقة؟ يوحّد النقاط الثلاث بدل تكرار 15 سطرًا ثلاث مرات.

الأسطر 157–160

```
 final String asset;
 final Color tileColor;
 final String title;
 final String subtitle;
```

ماذا يفعل؟ الحقول الأربعة.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 162–164

```
 @override
 Widget build(BuildContext context) {
  return Row(
```

ماذا يفعل؟ `build` و`Row` بمسافة 14.

لماذا بهذه الطريقة؟ ترتيب الصف: مربع الأيقونة (يمين) ثم النص.

الأسطر 165–167

```
      spacing: 14,
      children: [
        IconTile(asset: asset, color: tileColor),
```

ماذا يفعل؟ نعرض `IconTile` بالحجم الافتراضي 48.

لماذا بهذه الطريقة؟ استعمال المكوّن المشترك.

الأسطر 168–176

```
        Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(title, style: AppTextStyles.body16Strong),
              Text(subtitle, style: AppTextStyles.caption13),
            ],
          ),
        ),
```

ماذا يفعل؟ `Expanded` فيه عمود: العنوان ثم الوصف بمحاذاة `start`.

لماذا بهذه الطريقة؟ `start` في RTL = اليمين فيبدأ النص من جهة الأيقونة. `Expanded` يمنع تجاوز النص الطويل عرض الشاشة.

الأسطر 177–180

```
      ],
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الصنف.

الأسطر 182–186

```
class _PageDots extends StatelessWidget {
  const _PageDots({required this.count, required this.current});

  final int count;
  final int current;
```

ماذا يفعل؟ ودجت `_PageDots`: العدد والحالي.

لماذا بهذه الطريقة؟ ودجت عام يصلح لأي عدد صفحات.

الأسطر 188–193

```
188   @override
189   Widget build(BuildContext context) {
190     return Row(
191       mainAxisAlignment: MainAxisAlignment.center,
192       spacing: 6,
193       children: [
```

ماذا يفعل؟ `Row` متمركز بمسافة 6 بين النقاط.

لماذا بهذه الطريقة؟ في RTL أول نقطة على اليمين وهي «الحالية» الأولى — يطابق Figma.

الأسطر 194–203

```
194         for (var i = 0; i < count; i++)
195           AnimatedContainer(
196             duration: const Duration(milliseconds: 200),
197             width: i == current ? 22 : 8,
198             height: 8,
199             decoration: BoxDecoration(
200               color: i == current ? AppColors.teal : AppColors.disabled,
201               borderRadius: BorderRadius.circular(4),
202             ),
203           ),
```

ماذا يفعل؟ حلقة تولّد النقاط: الحالية بعرض 22 واون teal، والبقية بعرض 8 ورمادي `disabled`، زوايا 4 وارتفاع 8.

لماذا بهذه الطريقة؟ `AnimatedContainer` يحرّك التغيّر عند تبدّل `current` لاحقًا بلا كود إضافي. `for` داخل القائمة تغني عن `map().toList()`.

الأسطر 204–207

```
204       ],
205     );
206   }
207 }
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الملف.